

ЛАБОРАТОРНА РОБОТА № 1

Вступ до ASP.NET Core

Мета: ознайомитися з основними принципами роботи .NET, навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти, набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використанням middleware.

Завдання 1. Встановлення інтегрованого середовища розробки (IDE) та необхідних компонентів

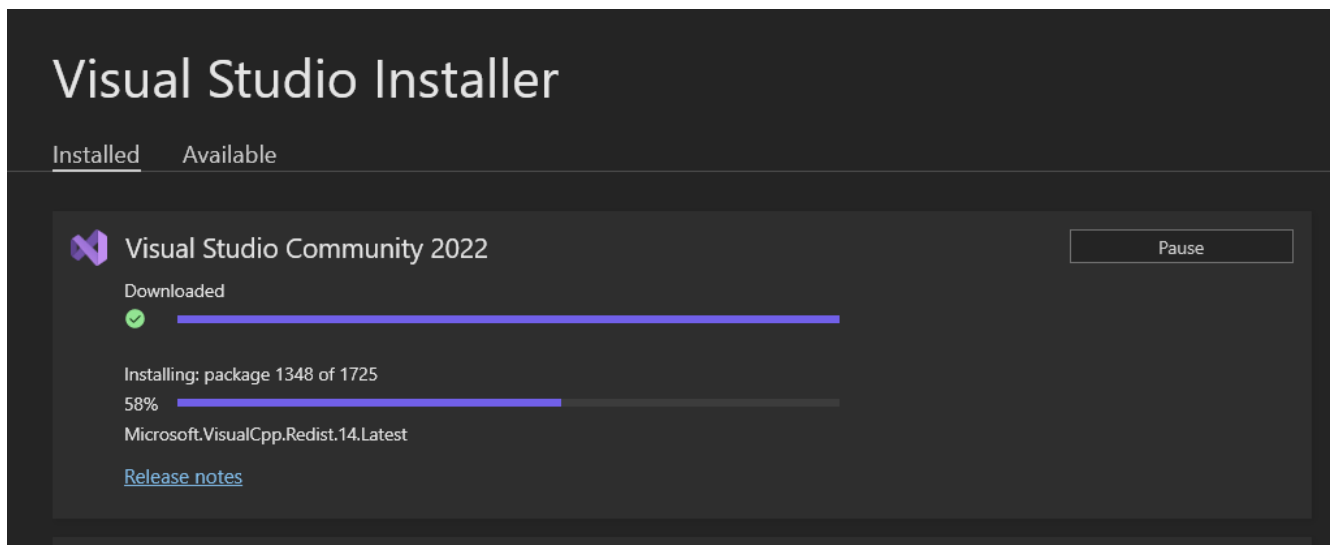


Рис. 1. Встановлення IDE

Завдання 2. Створення проектів

Частина 1.

1. Створіть проект для консольного додатку з назвою ConsoleToWeb з використанням [dotnet CLI](#)
2. Перетворіть створений консольний додаток на веб-додаток
3. Опишіть виконані кроки у звіті

					ДУ «Житомирська політехніка».25.121.11.000 – Лр1						
Змн.	Арк.	№ докум.	Підпис	Дата							
Розроб.		Зармбицький Є.Ю			Звіт з лабораторної роботи			Лім.	Арк.	Аркушів	
Перевір.		Українець М.О.								1	9
Керівник								ФІКТ Гр. ІПЗ-22-2			
Н. контр.											
Зав. каф.											

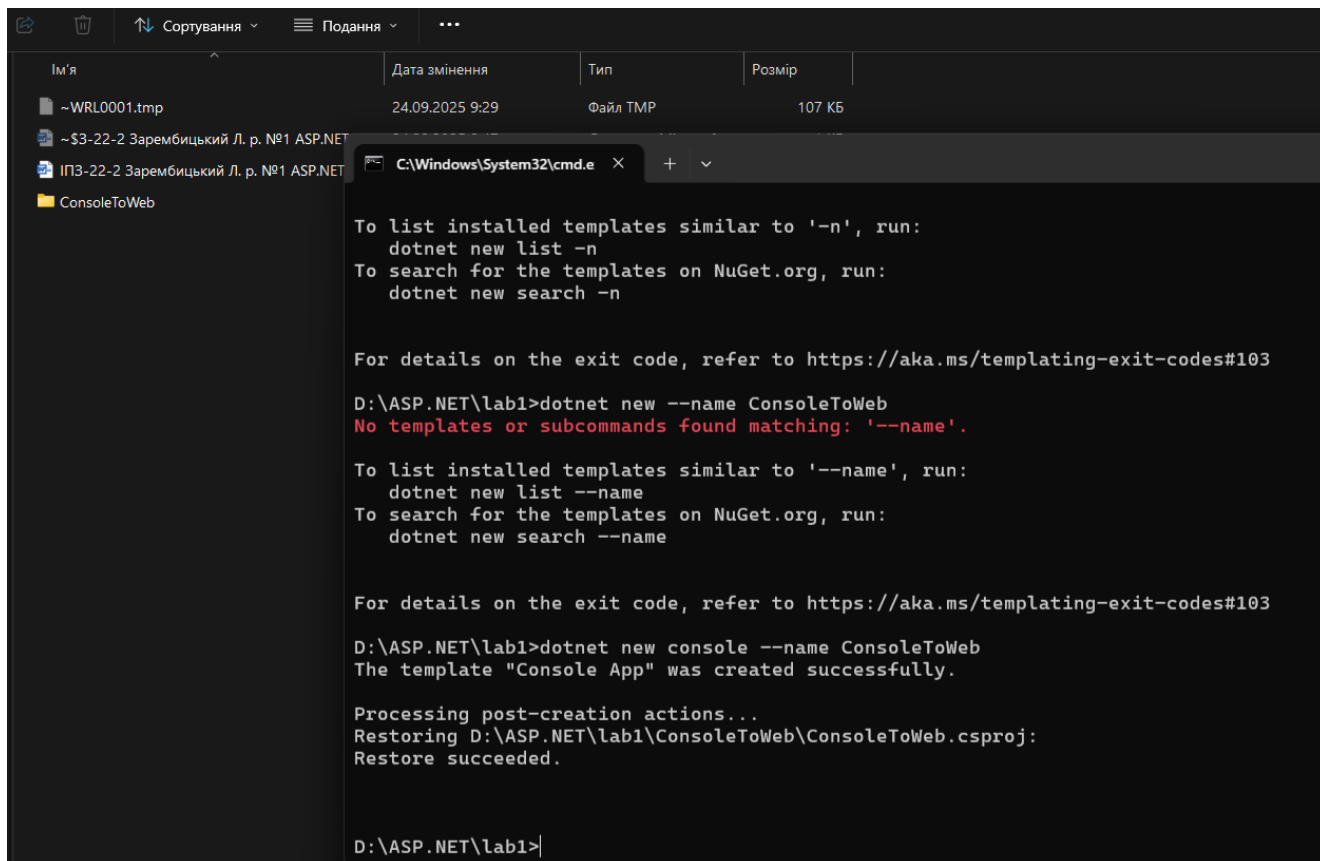


Рис. 2. Виконання команди dotnet new console --name

Для заміни на веб-додаток, змінюємо файл Program.cs

Лістинг

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
app.MapGet("/", () => "Now i'm a web app!");
```

```
app.Run();
```

Замінив Project.csproj sdk на Microsoft.Net.Sdk.Web

Частина 2.

1. Створіть ASP.NET WebAPI проект без авторизації з назвою WebFromCli з використанням [dotnet CLI](#)
2. Реалізуйте GET-ендпоінт “/who”, який повертатиме у відповідь ваше ім’я та прізвище
3. Реалізуйте GET-ендпоінт “/time”, який повертатиме у відповідь поточний час на сервері
4. Наведіть лістинг реалізованих обробників у звіті

		Зарембицький С.Ю.			ДУ «Житомирська політехніка».25.121.0.000 – Лр1	Арк.
		Українець М.О.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

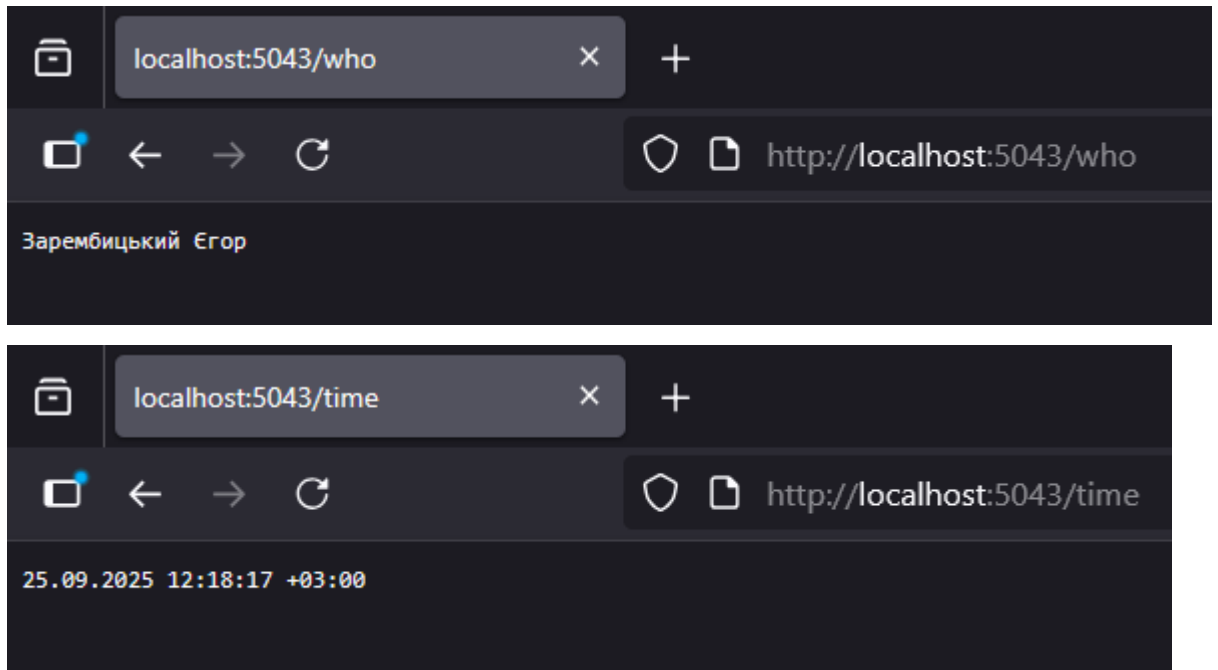


Рис. 3-4. Результат виконання програмного коду

Лістинг:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "Hello World!");
app.MapGet("/who", () => "Зарембицький Єгор");
app.MapGet("/time", () => GetTime());

string GetTime()
{
    DateTimeOffset time = DateTimeOffset.Now;
    return time.ToString();
}

app.Run();
```

Частина 3.

1. Створіть ASP.NET MVC проект будь-яким зручним способом.
2. Реалізуйте контролер з назвою LabController
3. В створеному контролері реалізуйте обробник /info, який повертатиме View з даними про номер лабораторної роботи, тему, мету та ім'я та прізвище виконавця в табличному вигляді
4. Дані для відображення передати з контролера

		Зарембицький Є.Ю.			ДУ «Житомирська політехніка».25.121.0.000 – Лр1	Арк.
		Українець М.О.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

№	1
Тема	Вступ до ASP.NET Core
Мета	Ознайомитися з основними принципами роботи .NET, навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти, набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використанням middleware.
Виконавець	Зарембицький Єгор

Рис. 5. Результат виконання

Лістинг:

```

using Microsoft.AspNetCore.Mvc;
using System.Reflection.Metadata.Ecma335;

namespace MVC.Controllers
{
    public class LabController: Controller
    {
        public IActionResult Info()
        {
            var LabData = new
            {
                Number = 1,
                Topic = "Вступ до ASP.NET Core",
                Purpose = "Ознайомитися з основними принципами роботи .NET," +
                    " навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти," +
                    " набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використан-
ням middleware.",
                Student = "Зарембицький Єгор",
            };
            return View(LabData);
        }
    }
}
@model dynamic
@{
    ViewData["Title"] = "Lab Info";
}

<div>
    <table>
        <tr>
            <td>№</td>
            <td>@Model.Number</td>
        </tr>
        <tr>
            <td>Тема</td>
            <td>@Model.Topic</td>
        </tr>
        <tr>
            <td>Мета</td>
            <td>@Model.Purpose</td>
        </tr>
        <tr>
            <td>Виконавець</td>
            <td>@Model.Student</td>
        </tr>
    </table>
</div>

```

Завдання 3. Робота з middleware

1. Ознайомтеся з [поняттям middleware](#) в ASP.NET
2. Створіть ASP.NET WebAPI проект з назвою MiddlewareSandbox

		Зарембицький Є.Ю.			ДУ «Житомирська політехніка».25.121.0.000 – Лр1	Арк.
		Українець М.О.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

3. Реалізуйте наступні middleware

а. Реалізуйте middleware, яке рахує кількість запитів до сервера і повертає це число у відповіді, наприклад: The amount of processed requests is X.

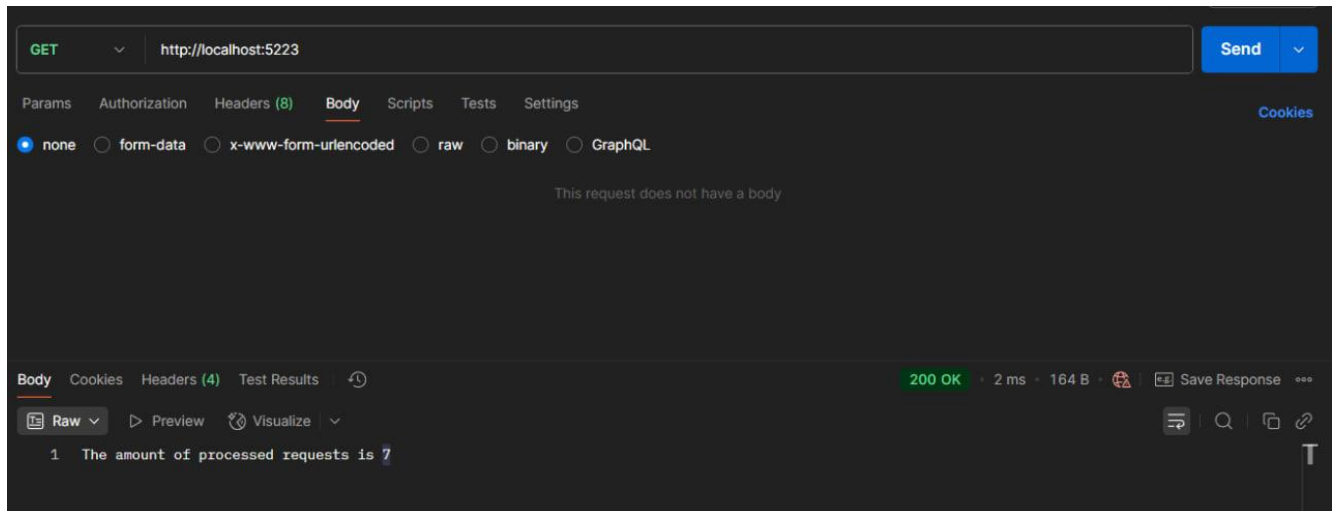


Рис. 6. Результат роботи програмного коду

Лістинг:

```
namespace MiddlewareSandbox
{
    public class RequestCounterMiddleware
    {
        private readonly RequestDelegate _next;
        private static int _requestCount = 0;

        public RequestCounterMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            int current = Interlocked.Increment(ref _requestCount);

            context.Response.ContentType = "text/plain";
            await context.Response.WriteAsync($"The amount of processed requests is {current}");
        }
    }
}
```

б. Створіть middleware, яке аналізує параметри рядка запиту. Якщо в рядку запиту (query string) присутній параметр “custom”, то повертати у відповідь “You’ve hit a custom middleware!”, інакше пропускати запит далі. Приклад запиту зображено на рисунку 3.1.

		Зарембицький С.Ю.			ДУ «Житомирська політехніка».25.121.0.000 – Лр1	Арк.
		Українець М.О.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

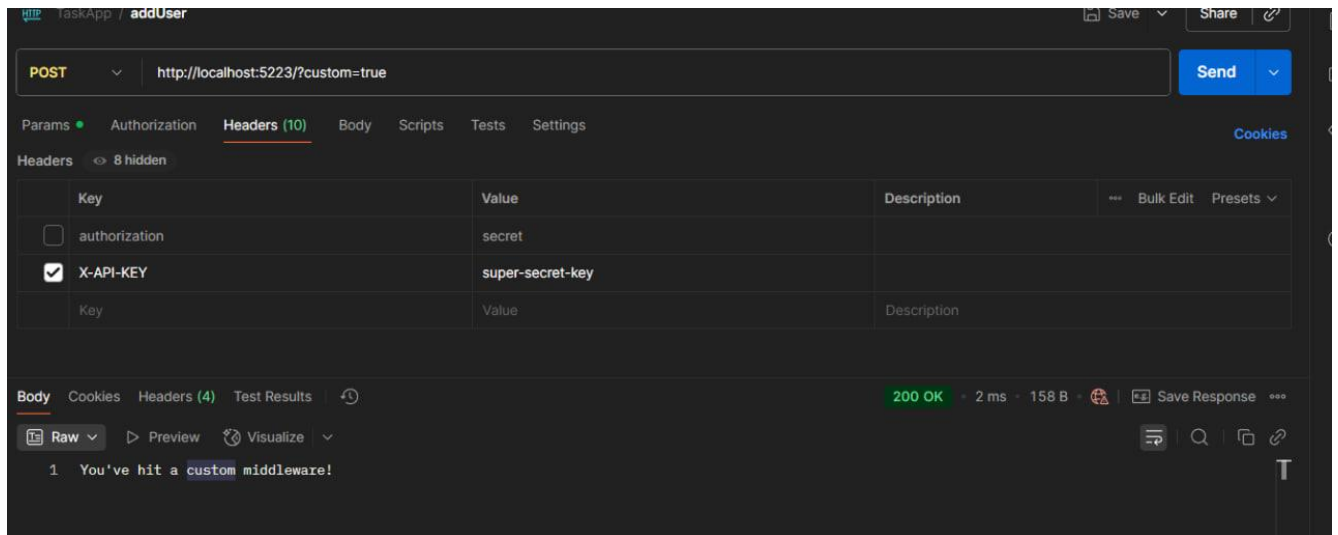


Рис. 7. Результат роботи програмного коду

Лістинг:

```
namespace MiddlewareSandbox
{
    public class CustomMiddleware
    {
        private readonly RequestDelegate _next;

        public CustomMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            if (context.Request.Query.ContainsKey("custom"))
            {
                context.Response.ContentType = "text/plain";
                await context.Response.WriteAsync("You've hit a custom middleware!");
            }
            else
            {
                await _next(context);
            }
        }
    }
}
```

с. Створіть middleware, яке логуватиме у консоль інформацію про всі запити. В логах відображати метод запиту (GET, POST тощо) та його шлях (/user, /product тощо). Для перевірки POST та інших методів запиту можна скористатися Postman або іншою подібною утилітою.

```

D:\ASP.NET\lab1\Middleware! x + v
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: https://localhost:7039
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: http://localhost:5223
info: Microsoft.Hosting.Lifetime[0]
  Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
  Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
  Content root path: D:\ASP.NET\lab1\MiddlewareSandbox\MiddlewareSandbox
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] PATCH /apikey/
[LOG] POST /
[LOG] POST /

```

Рис. 8. Результат роботи програмного коду

Лістинг:

```

namespace MiddlewareSandbox
{
    public class LoggingMiddleware
    {
        private readonly RequestDelegate _next;

        public LoggingMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            Console.WriteLine($"[LOG] {context.Request.Method} {context.Request.Path}");

            await _next(context);
        }
    }
}

```

d. Реалізуйте middleware, яке перевіряє наявність API-ключа у заголовках запиту. Для цього слід Перевіряти, чи є в запиті заголовок X-API-KEY. Якщо ключ неправильний або відсутній, повертати 403 Forbidden, не передавати запит для виконання далі. Якщо ключ в заголовку запиту співпадає з тим, який заданий на сервері, передавати запит далі. Для додавання відповідного запиту можна скористатися Postman або подібними утилітами.

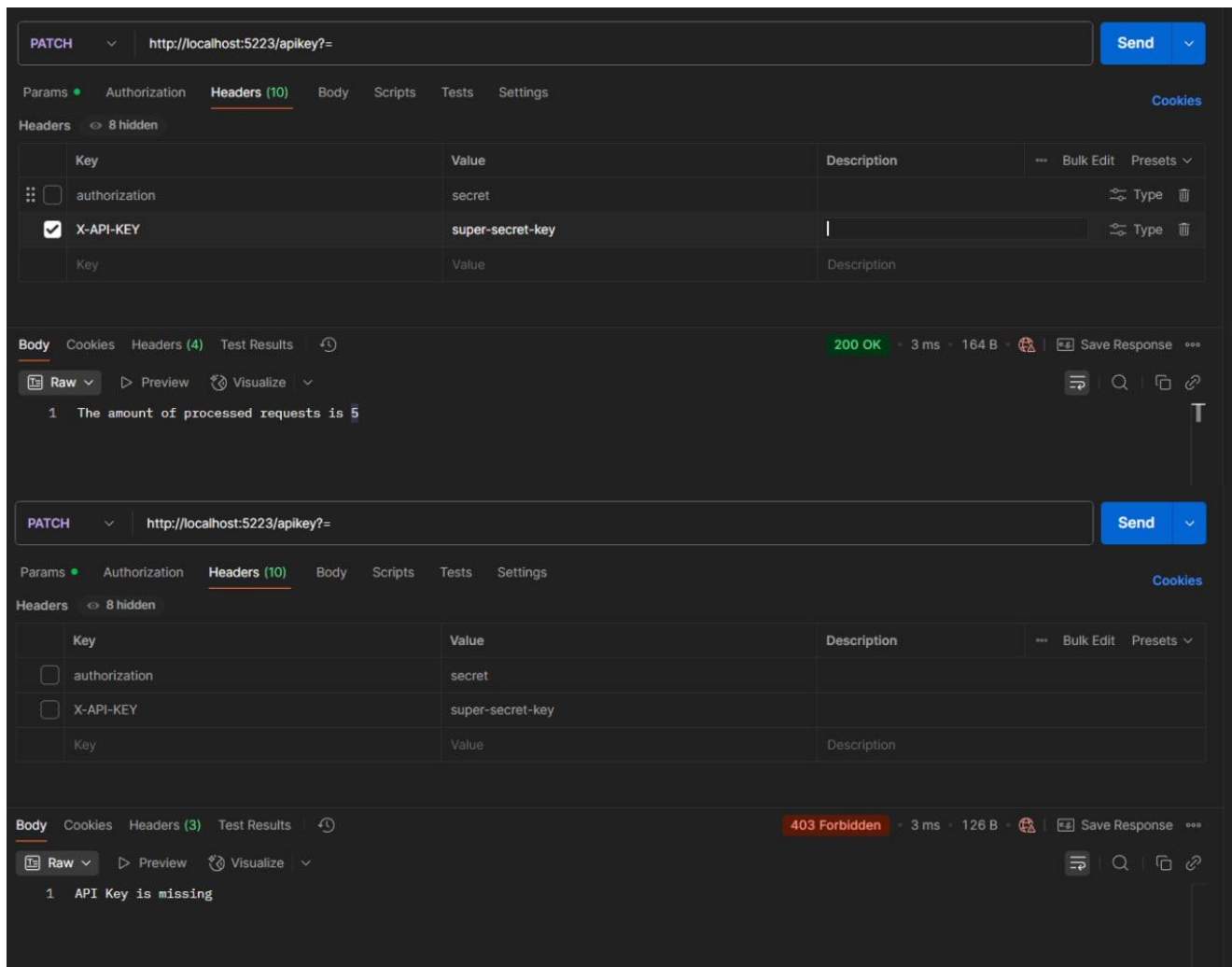


Рис. 9-10. Результат роботи програмного коду

Лістинг:

```
namespace MiddlewareSandbox
{
    public class ApiKeyMiddleware
    {
        private readonly RequestDelegate _next;
        private const string ApiKeyHeaderName = "X-API-KEY";

        private const string ServerApiKey = "super-secret-key";

        public ApiKeyMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            if (!context.Request.Headers.TryGetValue(ApiKeyHeaderName, out var extractedApiKey))
            {
                context.Response.StatusCode = StatusCodes.Status403Forbidden;
                await context.Response.WriteAsync("API Key is missing");
                return;
            }

            if (!ServerApiKey.Equals(extractedApiKey))
            {
                context.Response.StatusCode = StatusCodes.Status403Forbidden;
            }
        }
    }
}
```

		Зарембичський С.Ю.			ДУ «Житомирська політехніка».25.121.0.000 – Лр1	Арк.
		Українець М.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		8


```

        await context.Response.WriteAsync("Invalid API Key");
        return;
    }

    await _next(context);
}
}
}

```

Висновок: я ознайомився з основними принципами роботи .NET, навчився налаштовувати середовище розробки та встановлювати необхідні компоненти, набув навичок створення рішень та проектів різних типів, набув навичок обробки запитів з використанням middleware.

		Зарембицький С.Ю.			ДУ «Житомирська політехніка».25.121.0.000 – Лр1	Арк.
		Українець М.О.				9
Змн.	Арк.	№ докум.	Підпис	Дата		